

Настройка тестового стенда Med Doctor Assignment Service

Med Doctor Assignment Service. Версия документации: 2026-04-29.

Содержание

Настройка тестового стенда Med Doctor Assignment Service

1. Назначение тестового стенда
2. Минимальный состав стенда
3. Данные, которые нужно собрать перед настройкой
4. Рекомендуемая структура каталога стенда
5. Базовый профиль тестового стенда
6. Профили проверки
 - 6.1. Профиль A: безопасная проверка без websocket и без med-robot
 - 6.2. Профиль B: проверка websocket-корреляции без мутаций
 - 6.3. Профиль C: проверка med-robot без реальных мутаций
 - 6.4. Профиль D: ограниченная реальная мутация на тестовом отделении
7. Порядок первичной настройки
8. Команды запуска
 - 8.1. Linux/systemd-free запуск
 - 8.2. Windows/PowerShell запуск
 - 8.3. Maven запуск из исходников
9. Smoke test после старта
10. Контрольные строки логов
11. Критерии готовности тестового стенда
12. Частые ошибки настройки
13. Что приложить к отчету о проверке стенда

Документ предназначен для внедренца, разработчика, тестировщика и технической поддержки. Он описывает безопасную конфигурацию тестового стенда, порядок первичной проверки, признаки корректной работы и типовые ошибки при интеграции с Orchestra 6 и med-robot.

1. Назначение тестового стенда

Тестовый стенд нужен не только для запуска jar-файла, но и для проверки всей цепочки назначения визита:

1. сервис авторизуется в Orchestra;
2. читает справочники отделений, очередей, услуг, рабочих профилей и точек обслуживания;
3. подключается к событиям Orchestra через SockJS/STOMP или работает по расписанию;
4. находит визиты в очереди «Врач не назначен»;
5. восстанавливает контекст врача по событиям посадки;
6. выбирает услугу локально или через med-robot;
7. выполняет `assign-service` и `transfer-visit` только после защитных проверок;
8. пишет диагностические сообщения, достаточные для разбора инцидента.

Ключевая цель стенда - проверить конфигурацию и REST-контракты без риска повредить production-очереди. Поэтому первый запуск всегда выполняется с `dry-run=true` и ограничением `allowed-branches`.

2. Минимальный состав стенда

Компонент	Обязателен	Проверка доступности	Примечание
Orchestra 6	да	GET /rest/..., GET /qsystem/rest/config/...	Источник очередей, визитов, рабочих профилей и событий.
БД Orchestra	косвенно	через работоспособность Orchestra	Сервис напрямую в БД не ходит.
med-robot	опционально	POST /prorobot/optimalqueue/{branchId}/service/{serviceId}	Можно отключить через <code>application.med-robot.enabled=false</code> .
Med Doctor Assignment Service	да	GET /health или лог старта	Java 8, Micronaut 3.5.2.
Рабочее место врача	желательно	события <code>USER_SERVICE_POINT_SESSION_START</code> , <code>SET_WORK_PROFILE</code>	Нужен для end-to-end сценария.
Тестовые визиты	да	очередь «Врач не назначен» содержит ожидаемые талоны	Лучше создавать отдельные тестовые талоны.

3. Данные, которые нужно собрать перед настройкой

Перед редактированием `application.yml` необходимо зафиксировать значения в таблице.

Значение	Где взять	Для чего используется
<code>orchestra.url</code>	адрес стенда Orchestra	REST и SockJS/STOMP подключения.
сервисный логин/пароль	администратор Orchestra	Basic Auth для REST и websocket handshake.
<code>branchId</code> тестового отделения	справочник отделений Orchestra	<code>branches-for-cache</code> , <code>allowed-branches</code> .
<code>id</code> очереди «Врач не назначен»	справочник очередей/настройки маршрута	<code>unknown-doctor-queue-id</code> .
<code>id source entry point</code>	справочник entry point / проверка transfer	<code>source-entry-point-id-by-branch</code> .
<code>id</code> точки обслуживания врача	события посадки/справочник service point	диагностика контекста врача.
<code>staffId</code> тестового врача	события посадки/пользователи Orchestra	диагностика и фильтрация логов.
<code>workProfileId</code> врача	событие <code>SET_WORK_PROFILE</code>	проверка доступных врачу очередей и услуг.
услуги маршрута тестового визита	карточка визита/REST details	проверка локального выбора и med-robot.
URL med-robot	стенд med-robot	внешний выбор очереди/услуги.

Практическое правило: если неизвестен `source entry point id`, нельзя включать `dry-run=false`. Ошибка `fromId` в `transfer-visit` может приводить к 500, 403 или к формально успешному ответу без ожидаемого перемещения визита.

4. Рекомендуемая структура каталога стенда

```
/opt/med-doctor-assignment/  
  app/  
    med-doctor-assignment-service.jar  
  config/  
    application-test.yml  
  logs/  
    med-doctor-assignment-service.log  
    med-doctor-assignment-service-http.log  
    med-doctor-assignment-service-busines.log  
  scripts/  
    start.sh  
    stop.sh  
    smoke-test.sh
```

На Windows для ручного стенда можно использовать аналогичную структуру, например:

```
D:\med-doctor-assignment\  
  app\  
  config\  
  logs\  
  scripts\
```

5. Базовый профиль тестового стенда

Ниже приведен безопасный стартовый профиль. Значения id нужно заменить на реальные значения стенда.

```
micronaut:
  server:
    port: 8085
  http:
    client:
      read-timeout: 60000ms

application:
  orchestra:
    url: http://orchestra-test.local:8080
    username: ${ORCHESTRA_USER:superadmin}
    password: ${ORCHESTRA_PASSWORD:change-me}
    common-rest-path: /rest
    configuration-rest-path: /qsystem/rest/config
    branches-for-cache: "1"
    replay-mutation-cookies: false
    reconnect-delay-ms: 30000

  websocket:
    enabled: false
    topic: /topic/event
    subscribed-events:
      - USER_SERVICE_POINT_SESSION_START
      - SET_WORK_PROFILE
    delay-before-reconnect-in-milliseconds: 10000
    send-cookies-in-handshake: false

  med-robot:
    enabled: false
    url: http://med-robot-test.local:8082
    optimal-service-path: /prorobot/optimalqueue/{branchId}/service/{serviceId}
    request-body-mode: TICKET_NUMBER_PLAIN_TEXT
    plain-text-policy: default
    error-handling-mode: FALLBACK_TO_LOCAL
    fallback-to-local-on-empty-response: true
    require-doctor-available-service: false
    require-known-queue: true

  assignment:
    enabled: true
    unknown-doctor-queue-id: 292
    max-visits-per-cycle: 3
    polling-enabled: true
    polling-cron: "0 */1 * * * ?"
    branch-lock-timeout-ms: 5000
    dry-run: true
    allowed-branches: [1]
    stale-cache-duration-seconds: 300
    event-deduplication-ttl-seconds: 120
    processed-visit-ttl-seconds: 120

    service-point-open-trigger-enabled: false
    set-work-profile-trigger-enabled: false
    work-profile-expanded-trigger-enabled: true
    user-service-point-session-start-trigger-enabled: true
    user-session-settle-window-ms: 2000
```

```
recheck-visit-before-transfer: true
abort-cycle-on-forbidden-mutation: true
treat-inactive-user-state-as-failure: true
treat-no-started-service-point-session-as-failure: true

source-entry-point-id-by-branch:
  1: 1

experimental-endpoints:
  enabled: true
  queue-visits-path: "/rest/entrypoint/branches/{branchId}/queues/{queueId}/visits/
full/"
  visit-details-path: "/rest/entrypoint/branches/{branchId}/visits/{visitId}/"
  visit-by-id-path: "/rest/entrypoint/branches/{branchId}/visits/{visitId}/"
  assign-service-path: "/rest/entrypoint/branches/{branchId}/visits/{visitId}/services/
{serviceId}/"
  transfer-visit-path: "/rest/entrypoint/branches/{branchId}/queues/{queueId}/visits/"
```

6. Профили проверки

6.1. Профиль А: безопасная проверка без websocket и без med-robot

Используется для первого запуска и проверки REST-контрактов Orchestra.

```
application:
  websocket:
    enabled: false
  med-robot:
    enabled: false
  assignment:
    dry-run: true
    polling-enabled: true
    max-visits-per-cycle: 3
    allowed-branches: [1]
```

Ожидаемый результат: сервис читает справочники, видит очередь «Врач не назначен», пишет в лог, какие визиты и услуги были бы обработаны, но не выполняет `assign-service` и `transfer-visit`.

6.2. Профиль В: проверка websocket-корреляции без мутаций

Включается после успешной проверки профиля А.

```

application:
  websocket:
    enabled: true
    subscribed-events:
      - USER_SERVICE_POINT_SESSION_START
      - SET_WORK_PROFILE
  assignment:
    dry-run: true
    polling-enabled: true
    service-point-open-trigger-enabled: false
    set-work-profile-trigger-enabled: false
    user-service-point-session-start-trigger-enabled: true
    work-profile-expanded-trigger-enabled: true

```

Ожидаемый результат: при посадке врача в логах появляется связка `USER_SERVICE_POINT_SESSION_START -> SET_WORK_PROFILE`, после чего сервис запускает цикл с `TriggerSource=USER_SESSION_READY`.

6.3. Профиль C: проверка med-robot без реальных мутаций

```

application:
  med-robot:
    enabled: true
    request-body-mode: TICKET_NUMBER_PLAIN_TEXT
    error-handling-mode: FALLBACK_TO_LOCAL
    fallback-to-local-on-empty-response: true
  assignment:
    dry-run: true

```

Ожидаемый результат: сервис вызывает med-robot по номеру талона, принимает `serviceId/queueId`, проверяет очередь в branch cache и пишет планируемое действие в лог.

6.4. Профиль D: ограниченная реальная мутация на тестовом отделении

Включается только после успешных профилей A-C.

```

application:
  assignment:
    dry-run: false
    allowed-branches: [1]
    max-visits-per-cycle: 1
    recheck-visit-before-transfer: true
    abort-cycle-on-forbidden-mutation: true

```

Ожидаемый результат: один тестовый визит получает назначенную услугу и переводится в очередь, выбранную локально или med-robot.

7. Порядок первичной настройки

1. Скопировать jar и внешний `application-test.yml` на стенд.

2. Указать `application.orchestra.url`, сервисную учетную запись и `branches-for-cache`.
3. Указать `unknown-doctor-queue-id` для тестового отделения.
4. Указать `allowed-branches` только с тестовым отделением.
5. Указать `source-entry-point-id-by-branch` для тестового отделения.
6. Оставить `dry-run=true`.
7. Отключить websocket на первом запуске: `application.websocket.enabled=false`.
8. Отключить med-robot на первом запуске: `application.med-robot.enabled=false`.
9. Запустить сервис и дождаться сообщений о refresh branch cache.
10. Создать или найти тестовый визит в очереди «Врач не назначен».
11. Проверить, что polling видит визит и пишет планируемые действия.
12. Включить med-robot, если он входит в тестовый сценарий.
13. Включить websocket и проверить посадку врача.
14. Только после этого включать `dry-run=false` с `max-visits-per-cycle=1`.

8. Команды запуска

8.1. Linux/systemd-free запуск

```
cd /opt/med-doctor-assignment
export ORCHESTRA_USER=superadmin
export ORCHESTRA_PASSWORD='***'
java -jar app/med-doctor-assignment-service.jar --micronaut.config.files=config/
application-test.yml
```

8.2. Windows/PowerShell запуск

```
cd D:\med-doctor-assignment
$env:ORCHESTRA_USER="superadmin"
$env:ORCHESTRA_PASSWORD="***"
java -jar .\app\med-doctor-assignment-service.jar --micronaut.config.files=.
\config\application-test.yml
```

8.3. Maven запуск из исходников

```
./mvnw clean test
./mvnw mn:run -Dmicronaut.config.files=./src/main/resources/application.yml
```

Для Windows:

```
.\mvnw.cmd clean test
.\mvnw.cmd mn:run -Dmicronaut.config.files=.\src\main\resources\application.yml
```


9. Smoke test после старта

Шаг	Что сделать	Ожидаемый результат
1	Проверить, что процесс запущен	порт 8085 слушает, процесс Java жив.
2	Проверить refresh кэша	в логах есть <code>Start cache refresh for branch</code> и <code>Finish cache refresh for branch</code> .
3	Проверить очередь	лог содержит <code>Visits in unknown-doctor queue ... count=....</code> .
4	Проверить dry-run	лог содержит план действий без реальных REST PUT/POST мутаций.
5	Посадить врача на тестовую точку	при websocket включен лог корреляции сессии и профиля.
6	Вызвать med-robot	при включенном роботе виден HTTP вызов и выбранные <code>serviceId/queueId</code> .
7	Включить один реальный цикл	один тестовый визит меняет состояние в Orchestra.
8	Проверить повторную защиту	тот же маршрутный шаг не обрабатывается повторно в пределах TTL.

10. Контрольные строки логов

Ищите следующие сообщения:

```
Refresh caches for configured branches
Start cache refresh for branch
Finish cache refresh for branch
Cannot resolve configured branches from Orchestra. Service will stay alive and retry later
Schedule Orchestra REST reconnect/cache bootstrap retry
Connecting to Orchestra websocket
Subscribed to
Start assignment cycle
Doctor ... available services=
Visits in unknown-doctor queue
Call med-robot optimal service
Visit ... matchFound=true
Dry-run enabled. Skip mutation
Assign service response
Transfer visit response
Finish assignment cycle
```

Если `Start assignment cycle` есть, но med-robot не вызывается, проверьте `request-body-mode`, наличие `ticketId`, текущую услугу визита, `route fingerprint` и `processed-visit-ttl-seconds`.

11. Критерии готовности тестового стенда

Стенд считается готовым для функционального тестирования, если выполнены все условия:

- сервис стартует без аварийного завершения при временной недоступности Orchestra;
- branch cache прогревается минимум для одного тестового отделения;
- очередь «Врач не назначен» читается и содержит ожидаемые тестовые визиты;
- `dry-run=true` показывает корректный план назначения;
- `USER_SERVICE_POINT_SESSION_START -> SET_WORK_PROFILE` коррелируется для тестового врача;
- med-robot, если включен, возвращает ожидаемые `serviceId` и `queueId`;
- при `dry-run=false`, `max-visits-per-cycle=1` один тестовый визит переводится в ожидаемую очередь;
- в логах нет массовых повторов 403/500;
- при ошибке Orchestra сервис не падает, а продолжает цикл восстановления.

12. Частые ошибки настройки

Симптом	Вероятная причина	Что исправить
Сервис не видит отделение	неверный <code>branches-for-cache</code> или права учетной записи	проверить id отделения и REST configuration API.
Очередь пустая, хотя в интерфейсе есть талоны	указан не тот <code>unknown-doctor-queue-id</code>	сверить именно queue id, а не service id.
med-robot не вызывается	<code>enabled=false</code> , нет <code>ticketId</code> , визит уже дедуплицирован	проверить <code>application.med-robot.*</code> и fingerprint визита.
med-robot возвращает услугу, но сервис ее отвергает	<code>require-doctor-available-service=true</code> или неизвестная очередь	на стенде чаще использовать <code>false</code> и проверить branch cache.
<code>assign-service</code> возвращает <code>INACTIVE</code>	операторская сессия еще не активирована	не включать сырой <code>SERVICE_POINT_OPEN</code> ; использовать корреляцию сессии и профиля.
<code>transfer-visit</code> падает из-за <code>fromId</code>	неверный source entry point id	заполнить <code>source-entry-point-id-by-branch</code> .
После первого 403 идет много одинаковых ошибок	выключен <code>abort-cycle-on-forbidden-mutation</code>	оставить <code>true</code> .
Сервис падает при недоступности Orchestra	используется устаревшая сборка или неверная обработка bootstrap	проверить наличие логов <code>retry</code> и <code>reconnect-delay-ms</code> .

13. Что приложить к отчету о проверке стенда

- версия jar и commit/hash сборки;
- полный `application-test.yml` без паролей;

- branch id, queue id, source entry point id;
- `staffId`, `servicePointId`, `workProfileId` тестового врача;
- ticket number и `visitId` тестового визита;
- фрагмент логов от старта до первого successful/dry-run цикла;
- ответ med-robot, если он участвовал;
- результат в интерфейсе Orchestra до и после теста;
- список отклонений и решение по каждому отклонению.